Execution Reports Documentation
Veeva Hybrid Automation Framework

Introduction

This document presents the execution reports generated for the Hybrid Automation Framework developed as part of the Veeva Systems assignment. The framework includes both UI and API automation using Selenium, RestAssured, Cucumber, and TestNG.

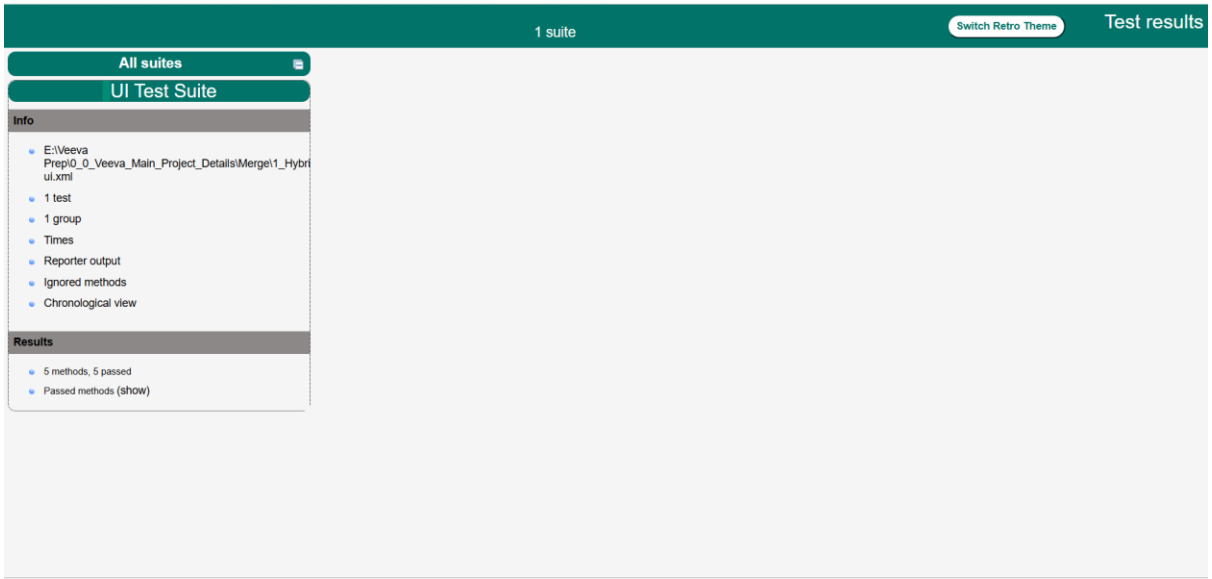
The screenshots included in this document represent different reporting tools used to track test execution results, failures, and overall test coverage.

1. TestNG Reports

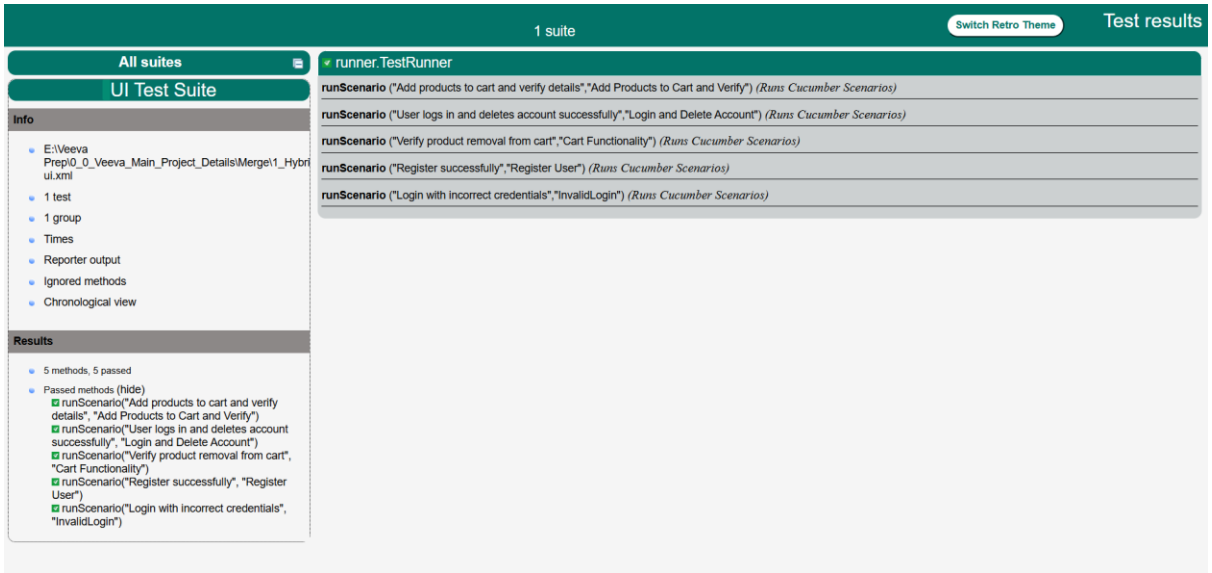
1.1 UI TestNG Reports

The TestNG report for UI automation provides a summary of all executed test cases. It shows the number of passed, failed, and skipped tests along with execution time. This helps in quickly understanding the overall status of UI test execution.

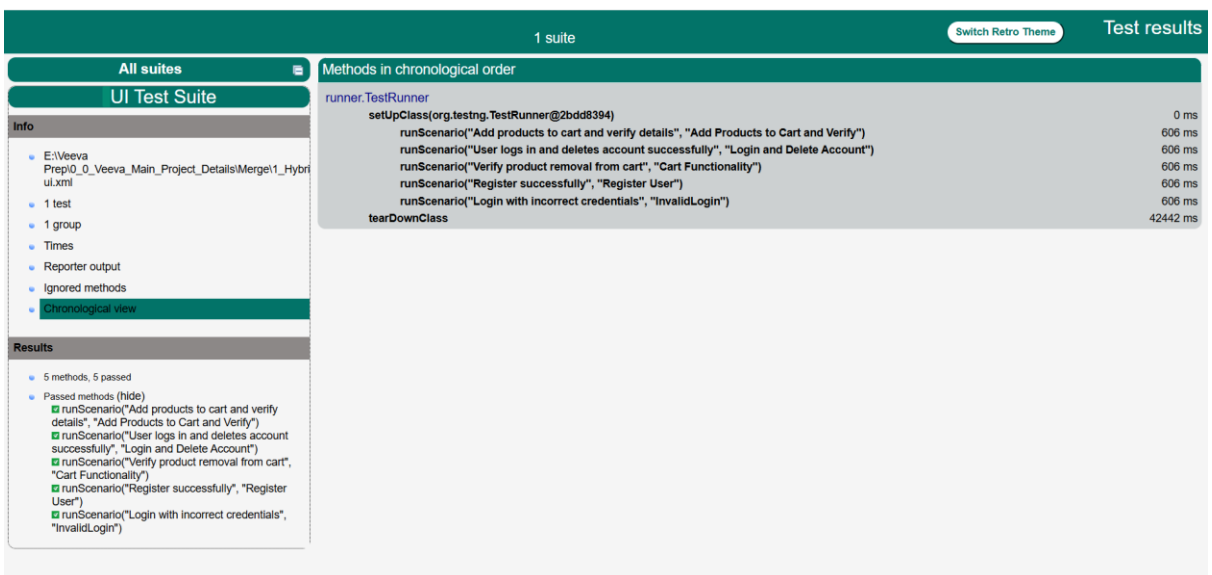
UI Test Suite results dashboard displaying execution details, with all five test methods passing successfully:



UI Test Suite execution report listing five Cucumber scenarios, all completed successfully without errors:



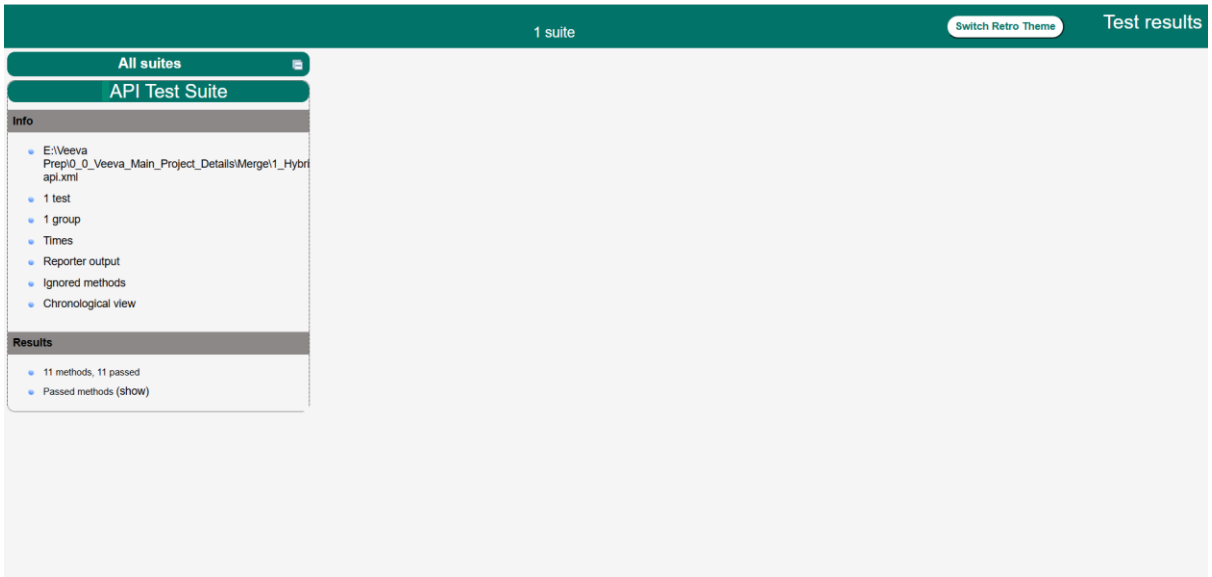
UI Test Suite report showing five executed methods, all passed, with chronological order and execution timings for each scenario:



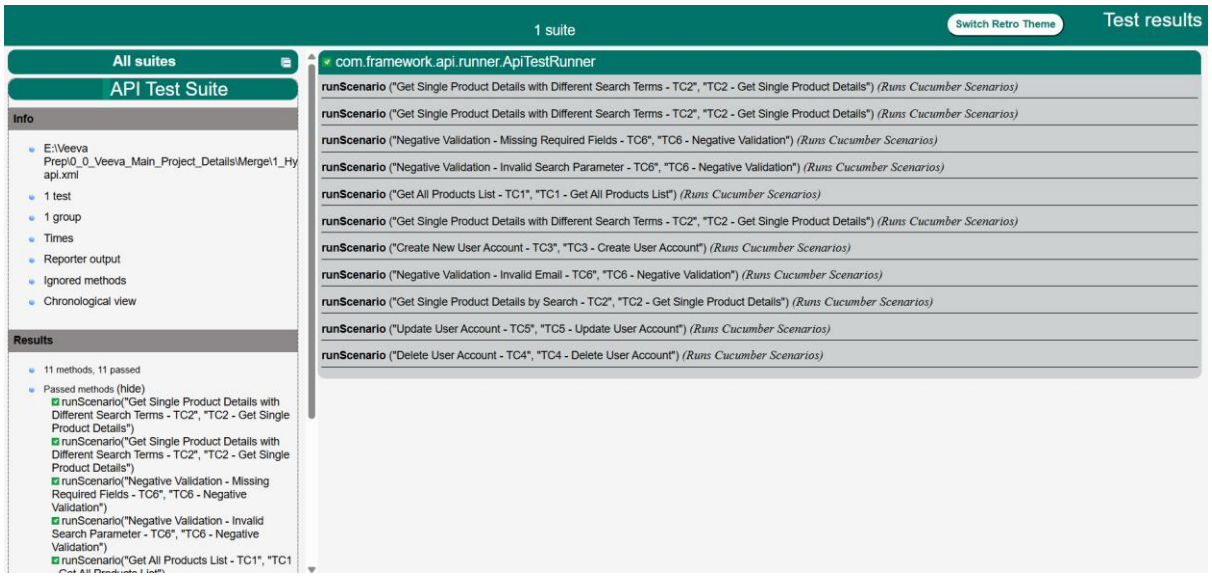
1.2 API TestNG Reports

The TestNG report for API automation includes execution details of API test cases. It validates endpoints, request responses, and ensures proper execution of all API scenarios.

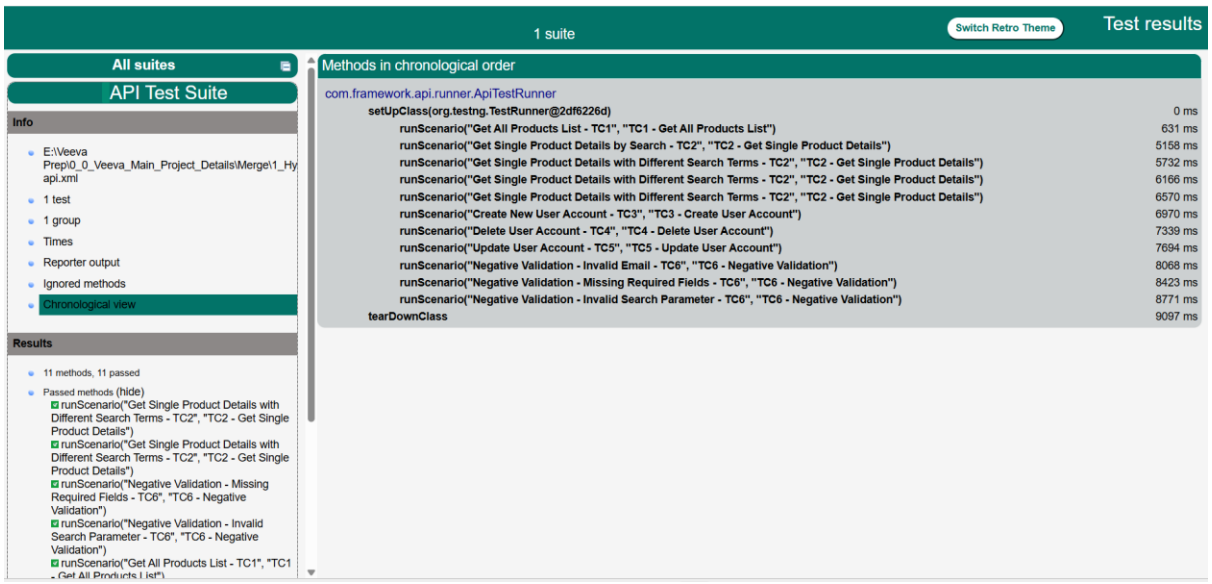
API Test Suite results dashboard showing execution details, with all eleven test methods passing successfully:



API Test Suite execution report displaying eleven Cucumber scenarios, all passed successfully, covering product details, account operations, and validation cases:



API Test Suite detailed execution log showing all eleven scenarios completed successfully, with timing data and step-wise validation for each API operation:

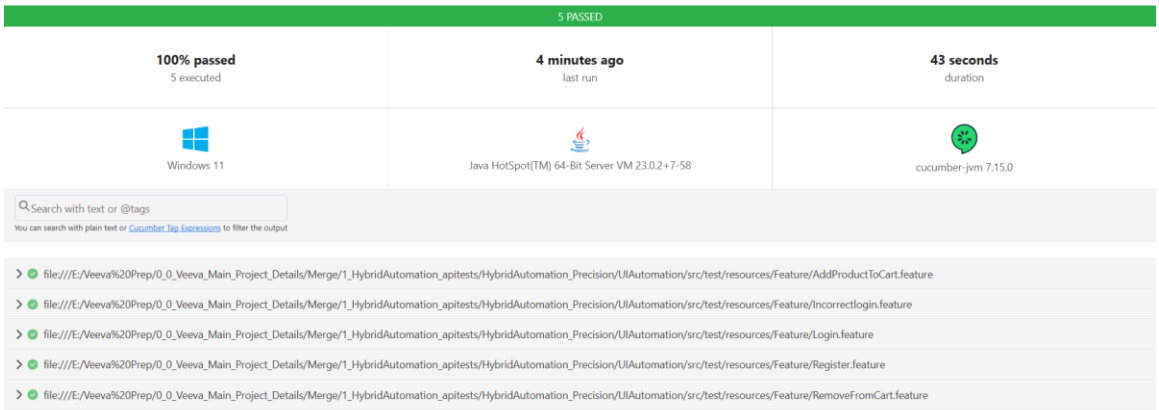


2. Cucumber Reports

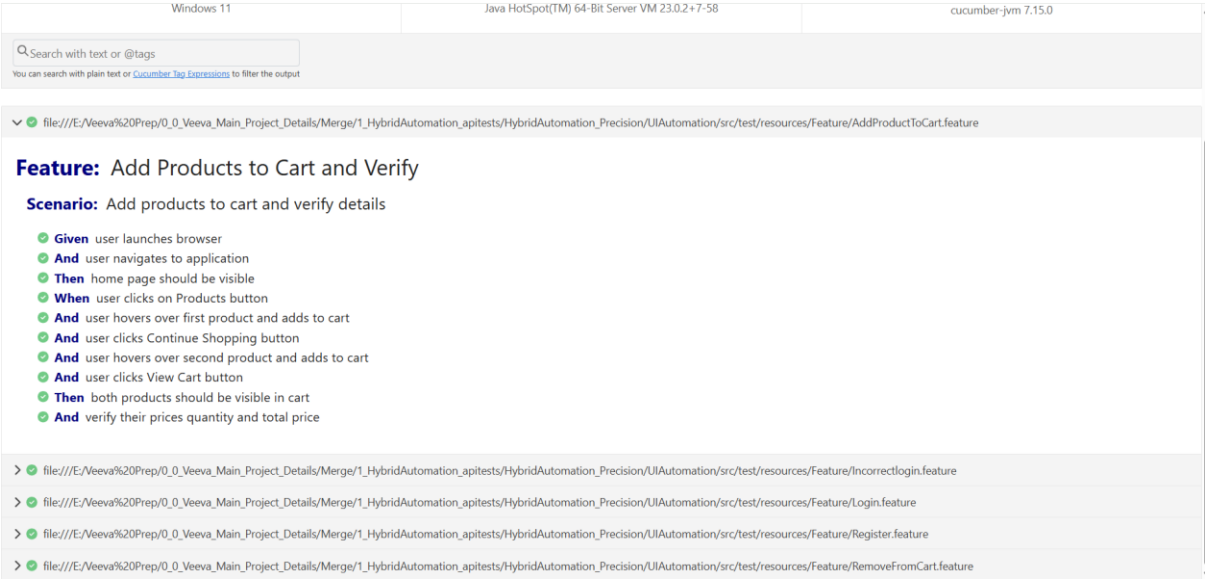
2.1 UI Cucumber Reports

Cucumber reports for UI automation display test results in a behavior-driven format. They show feature files, scenarios, and step-by-step execution results, making it easy to understand test flow.

Cucumber report summary dashboard confirming five automated UI feature tests all passed, with environment details and run statistics displayed:



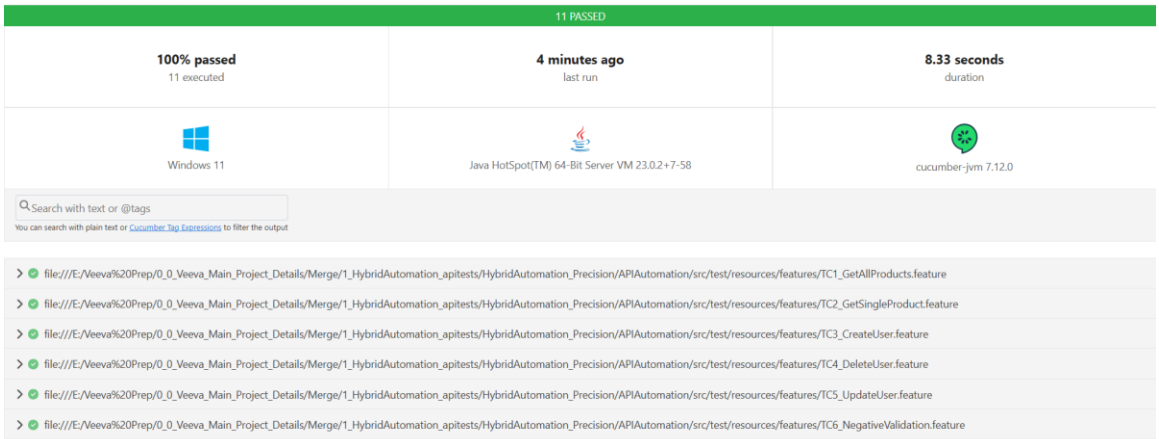
Cucumber feature file defining the ‘Add Products to Cart and Verify’ scenario, outlining step-by-step actions for validating cart functionality in the application:



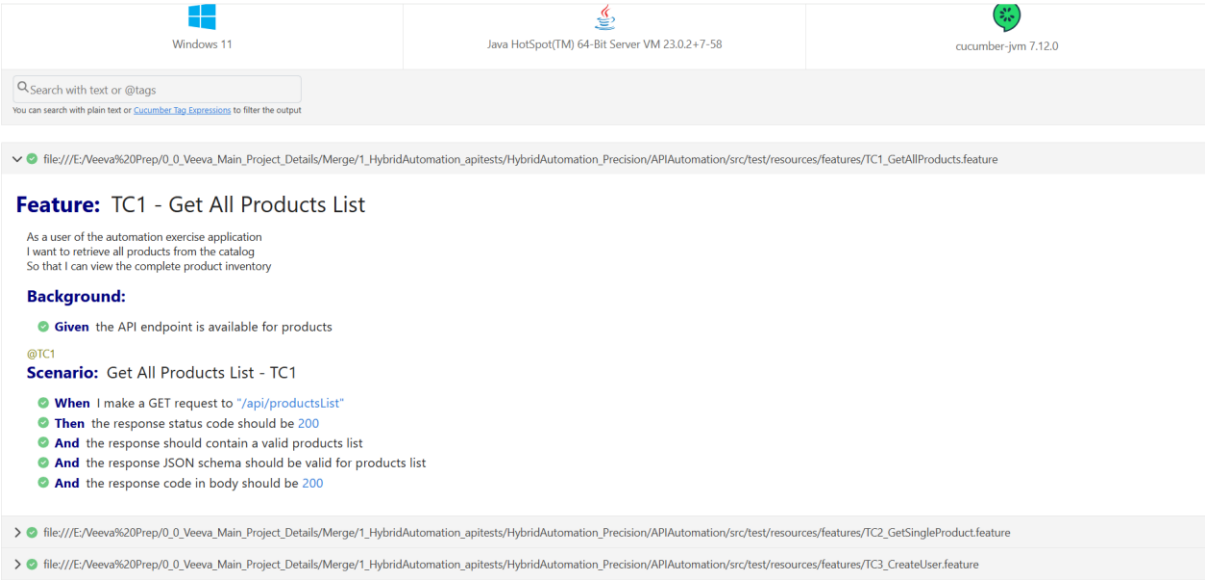
2.2 API Cucumber Reports

API Cucumber reports present the execution of API scenarios in a readable format. Each step (Given, When, Then) is validated and displayed clearly for better traceability.

Cucumber report summary confirming five UI scenarios executed successfully with 100% pass rate and environment details:



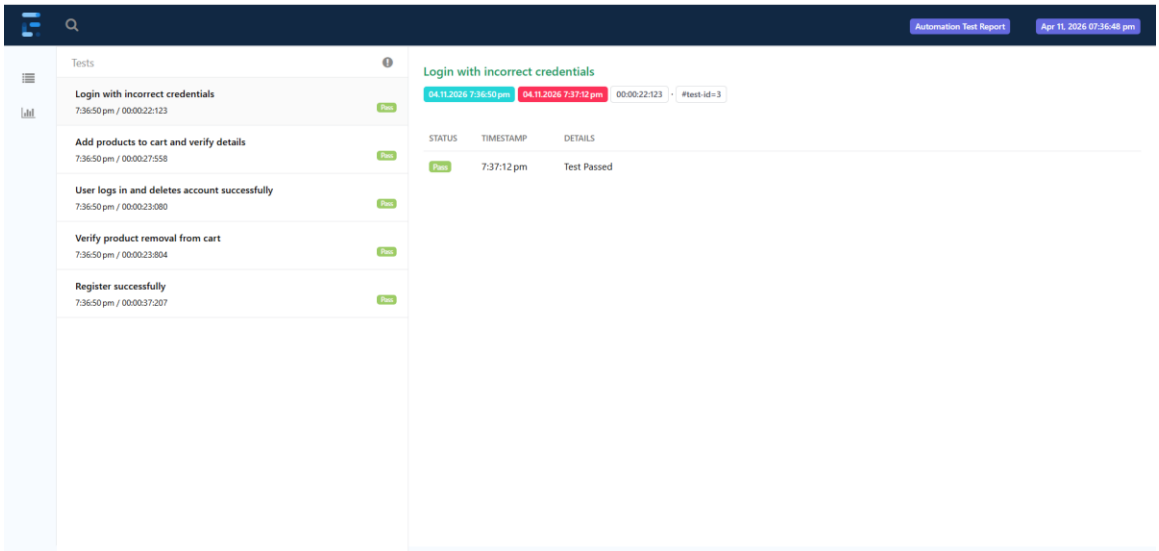
Cucumber feature file for API testing, defining the ‘Get All Products List’ scenario with step-wise validations for status code, JSON schema, and response content:



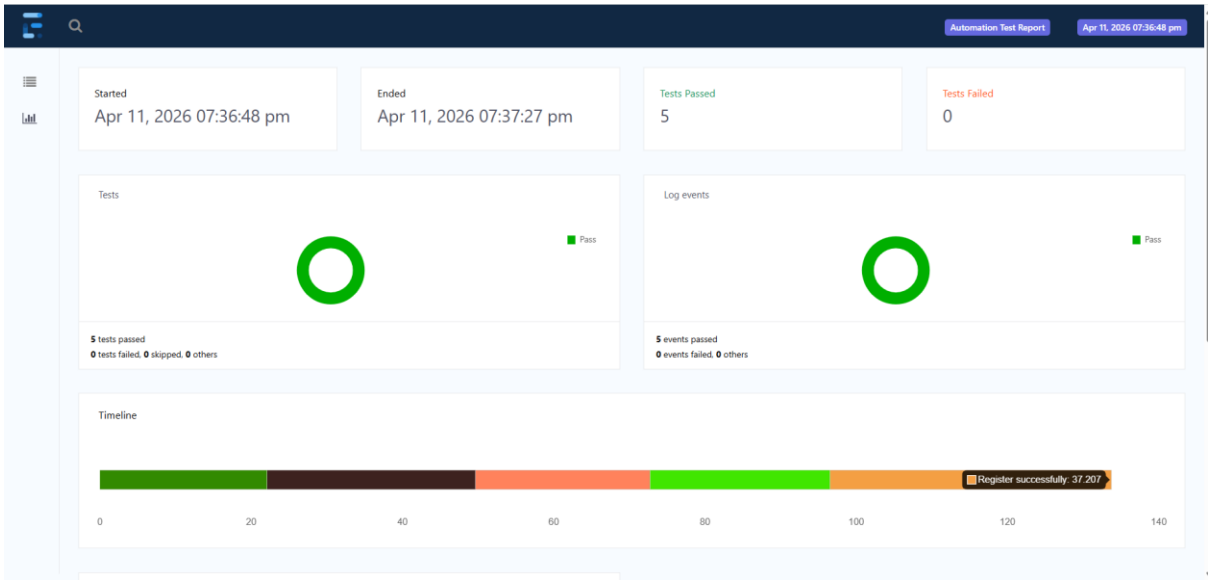
3. Extent Reports (UI)

Extent Reports provide a visually rich and detailed report for UI automation. It includes step-level logs, screenshots for failures, and overall execution status. This report is useful for debugging failed test cases.

Extent report dashboard displaying detailed execution results for UI scenarios, with timestamps, unique test IDs, and step-wise status confirmation for each case:



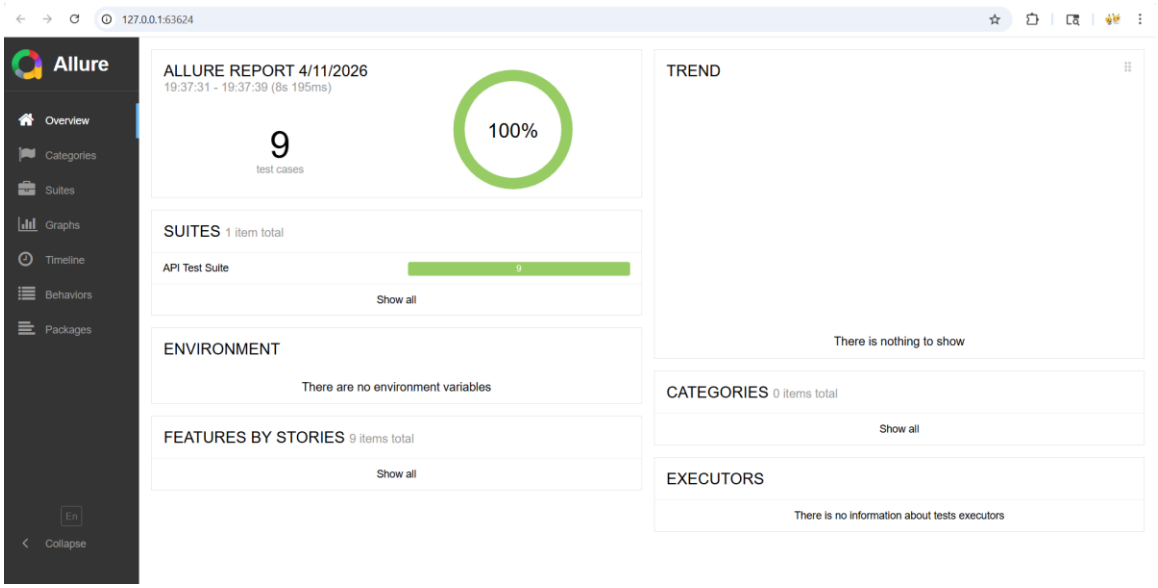
Extent report summary dashboard showing five tests executed with 100% pass rate, visual charts, and timeline view of scenario execution:



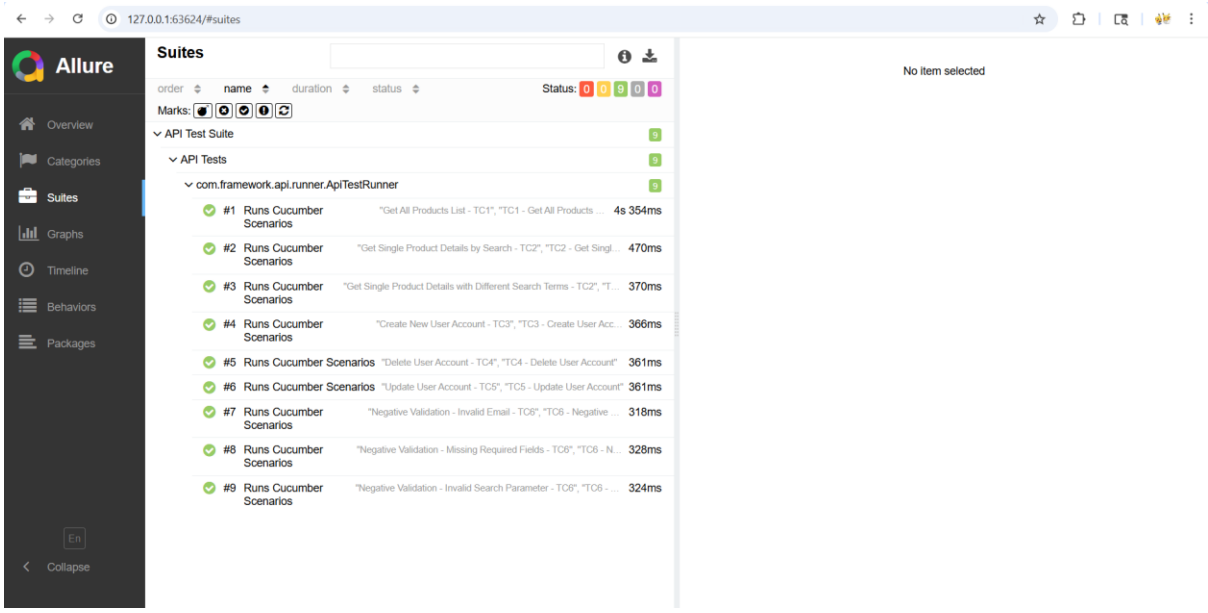
4. Allure Reports (API)

Allure Reports are used for API automation reporting. They provide detailed insights into test execution, including request/response data, execution steps, and failure analysis.

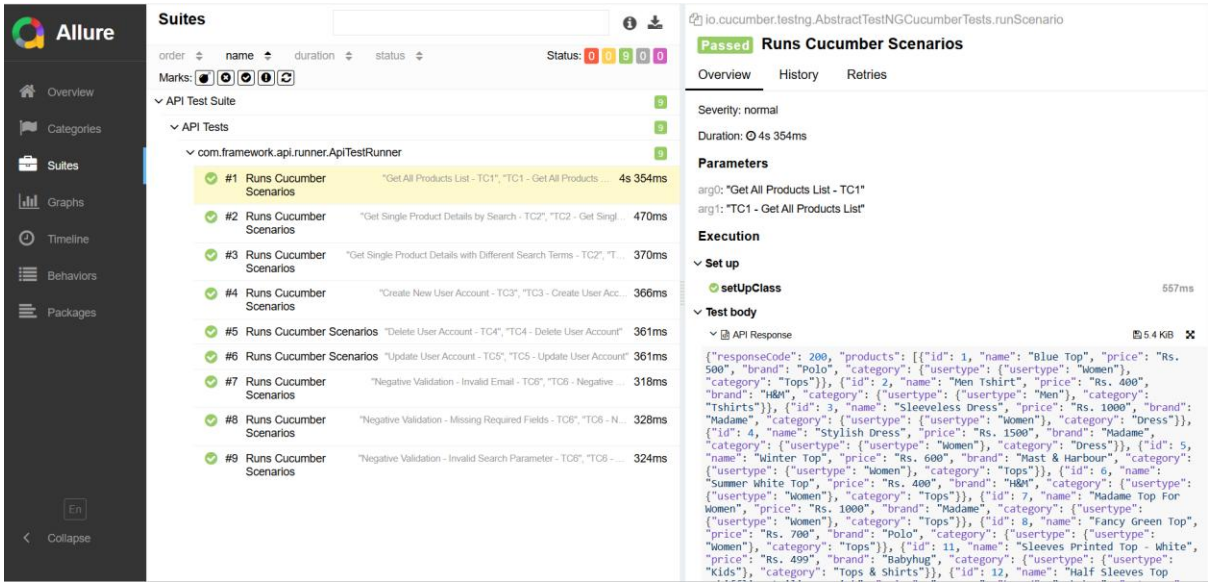
Allure report overview showing nine API test cases executed with 100% pass rate, suite summary, and structured navigation for detailed analysis:



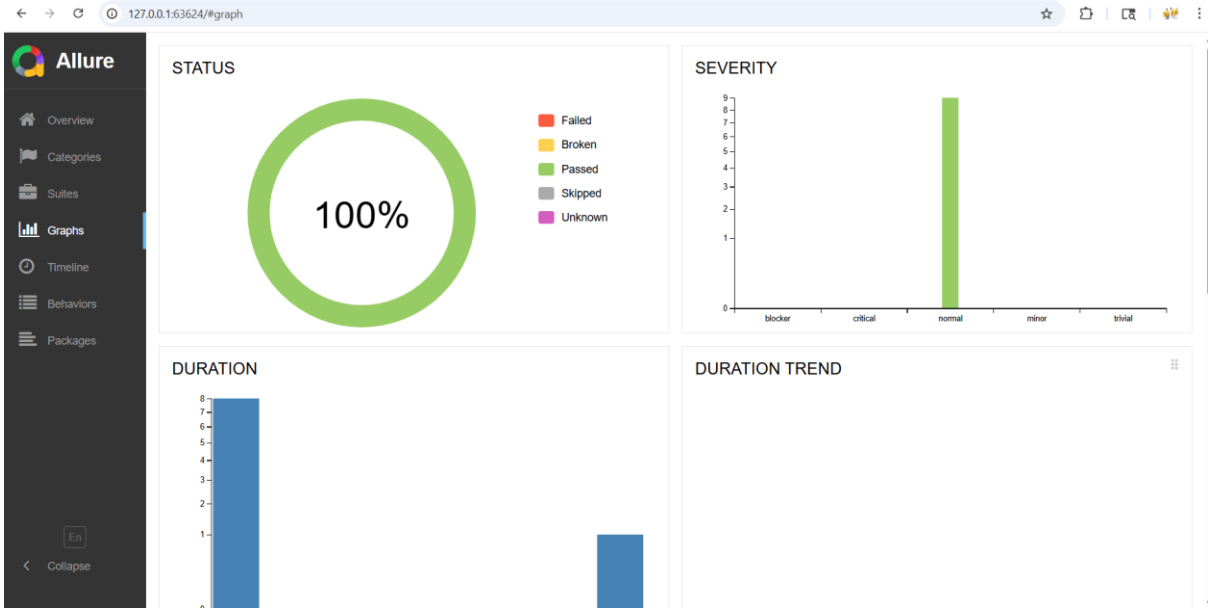
Allure report ‘Suites’ view displaying API Test Suite execution, with all scenarios passed successfully and detailed timing information for each case:



Allure report detailed test view showing API scenario execution with parameters, response validation, and JSON output confirming product data retrieval:



Allure report graphs view summarizing test execution with 100% pass rate, severity distribution, and duration analysis:

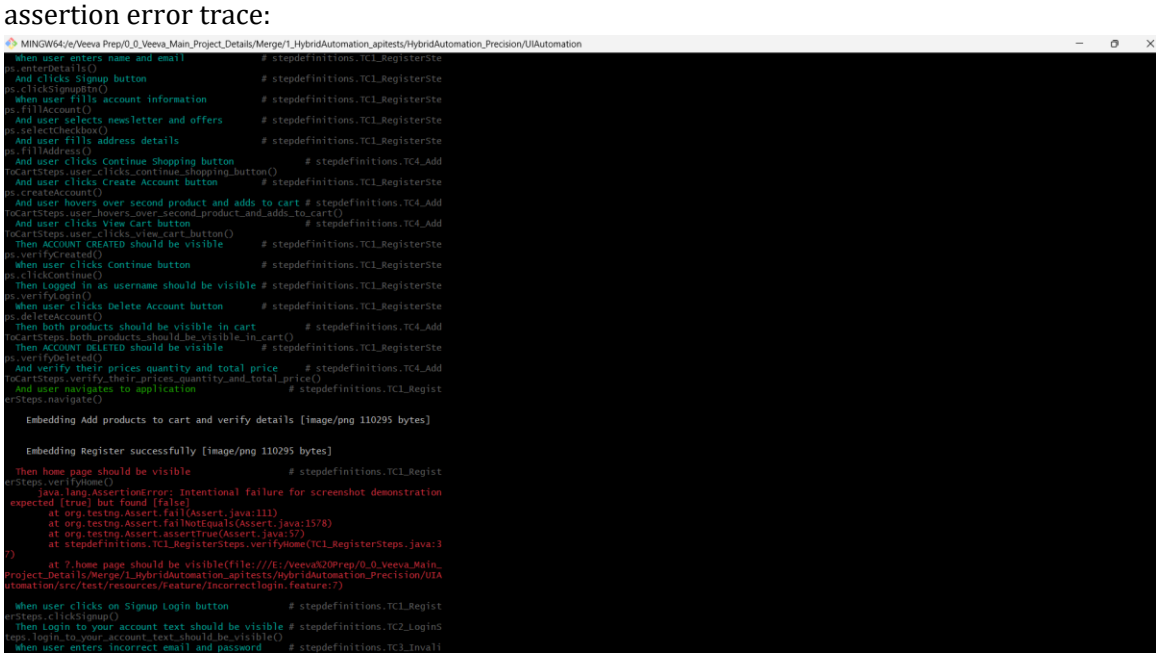


5. Failure Reports

5.1 UI Failure Reports (Extent)

When a UI test case fails, the framework automatically captures a screenshot and logs the failure in the Extent Report. This helps in identifying the exact point of failure and debugging the issue efficiently.

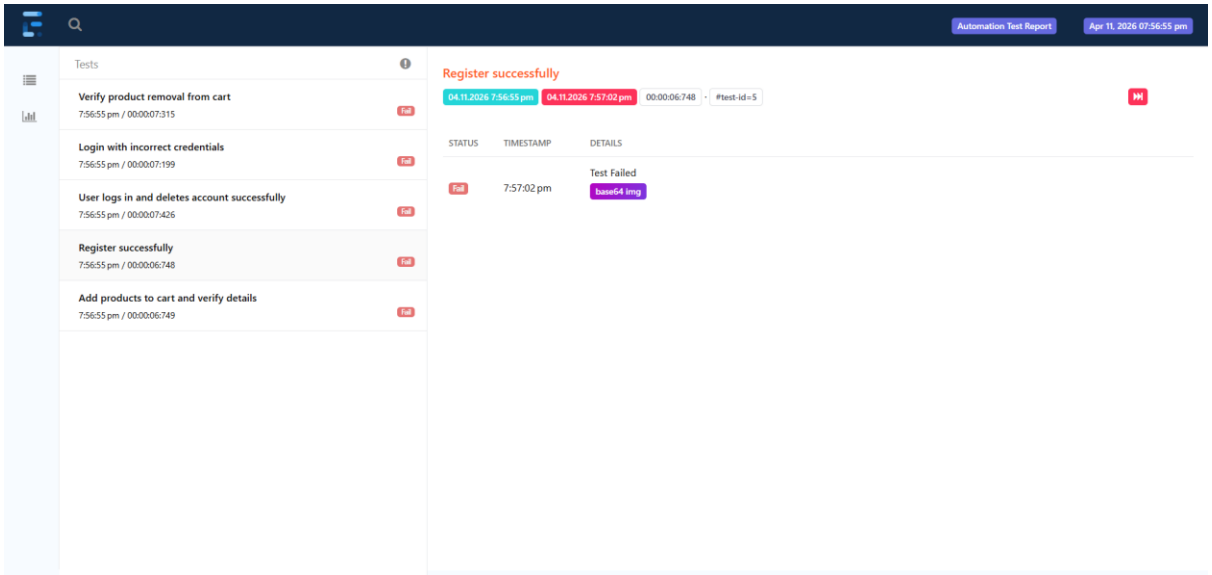
Git Bash console log showing Cucumber test execution with one failed UI scenario and assertion error trace:



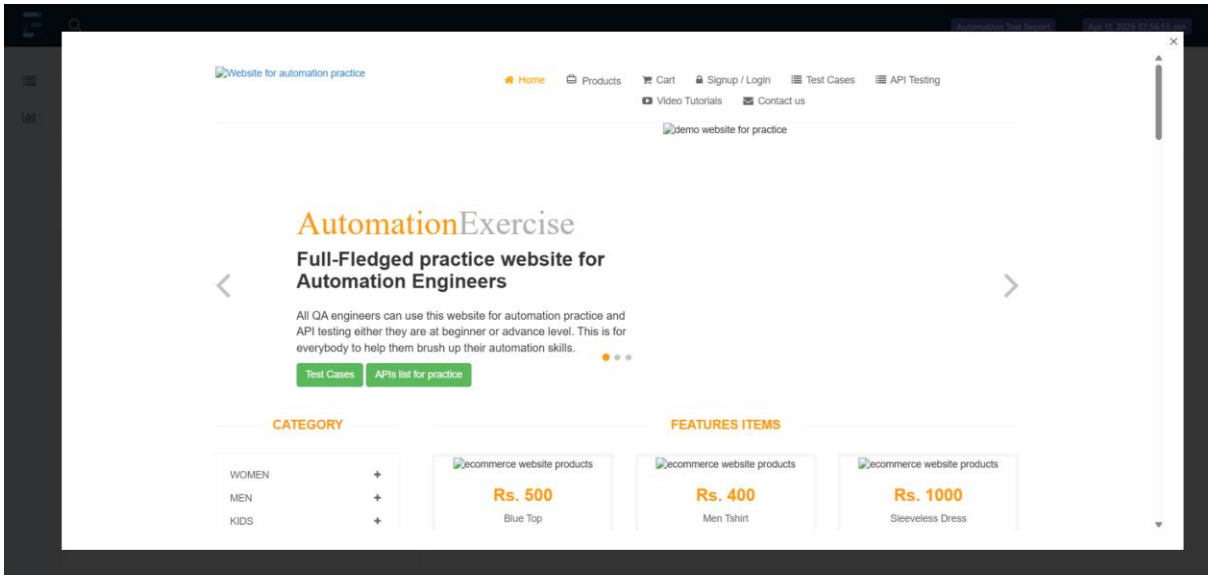
Git Bash terminal output showing Maven–TestNG UI test execution with intentional failures for screenshot demonstration, including stack traces from UIA_RegisterSteps.java:

```
        at stepdefinitions.TC1_RegisterSteps.verifyHome(TC1_RegisterSteps.java:3
7)
        at 7.home page should be visible(file:///E:/Veeva%20Prep/0_0_Veeva_Main_
Project_Details/Merge/1_HybridAutomation_apitests/HybridAutomation_Precision/UIA
utomation/src/test/resources/Feature/Login.feature:7)

[ERROR] runner.TestRunner.runScenario["Verify product removal from cart", "Cart
Functionality"](4)  Time elapsed: 13.88 s  <<< FAILURE!
java.lang.AssertionError: Intentional failure for screenshot demonstration expec
ted [true] but found [false]
        at org.testng.Assert.fail(Assert.java:111)
        at org.testng.Assert.failNotEquals(Assert.java:1578)
        at org.testng.Assert.assertTrue(Assert.java:57)
        at stepdefinitions.TC1_RegisterSteps.verifyHome(TC1_RegisterSteps.java:3
7)
        at 7.home page should be visible(file:///E:/Veeva%20Prep/0_0_Veeva_Main_
Project_Details/Merge/1_HybridAutomation_apitests/HybridAutomation_Precision/UIA
utomation/src/test/resources/Feature/RemoveFromCart.feature:7)

[ERROR] runner.TestRunner.runScenario["Register successfully", "Register User"](
5)  Time elapsed: 13.881 s  <<< FAILURE!
java.lang.AssertionError: Intentional failure for screenshot demonstration expec
ted [true] but found [false]
        at org.testng.Assert.fail(Assert.java:111)
        at org.testng.Assert.failNotEquals(Assert.java:1578)
        at org.testng.Assert.assertTrue(Assert.java:57)
        at stepdefinitions.TC1_RegisterSteps.verifyHome(TC1_RegisterSteps.java:3
7)
        at 7.home page should be visible(file:///E:/Veeva%20Prep/0_0_Veeva_Main_
Project_Details/Merge/1_HybridAutomation_apitests/HybridAutomation_Precision/UIA
utomation/src/test/resources/Feature/Register.feature:7)

[INFO]
[INFO] Results:
[INFO]
[ERROR] Failures:
[ERROR]   TestRunner.runScenario Intentional failure for screenshot demonstratio
n expected [true] but found [false]
[ERROR]   TestRunner.runScenario Intentional failure for screenshot demonstratio
n expected [true] but found [false]
[ERROR]   TestRunner.runScenario Intentional failure for screenshot demonstratio
n expected [true] but found [false]
[ERROR]   TestRunner.runScenario Intentional failure for screenshot demonstratio
n expected [true] but found [false]
[ERROR]   TestRunner.runScenario Intentional failure for screenshot demonstratio
n expected [true] but found [false]
[INFO]
[ERROR] Tests run: 5, Failures: 5, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD FAILURE
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  17.776 s
[INFO] Finished at: 2026-04-11T19:57:04+05:30
[INFO] ------------------------------------------------------------------------
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire-plugin:3.
0.0-M7:test (default-test) on project UIAutomation: There are test failures.
```

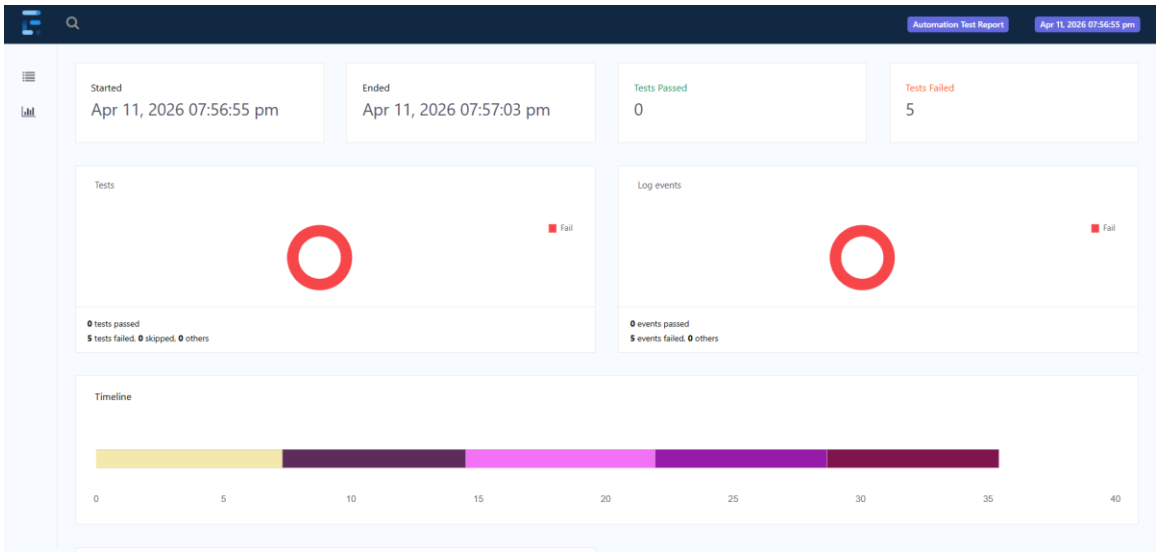
Extent report dashboard highlighting a failed UI scenario ('Register successfully'), with timestamp, failure status, and embedded screenshot evidence for debugging:



Screenshot of the AutomationExercise application captured at failure point, automatically embedded in Extent report for debugging and traceability:



Extent report summary dashboard showing five UI tests executed with 100% failure rate, visual charts, and timeline view of scenario execution:



5.2 API Failure Reports (Allure)

API failures are recorded in Allure Reports with detailed logs. These logs include request payloads, response data, and error messages, helping in quick issue resolution.

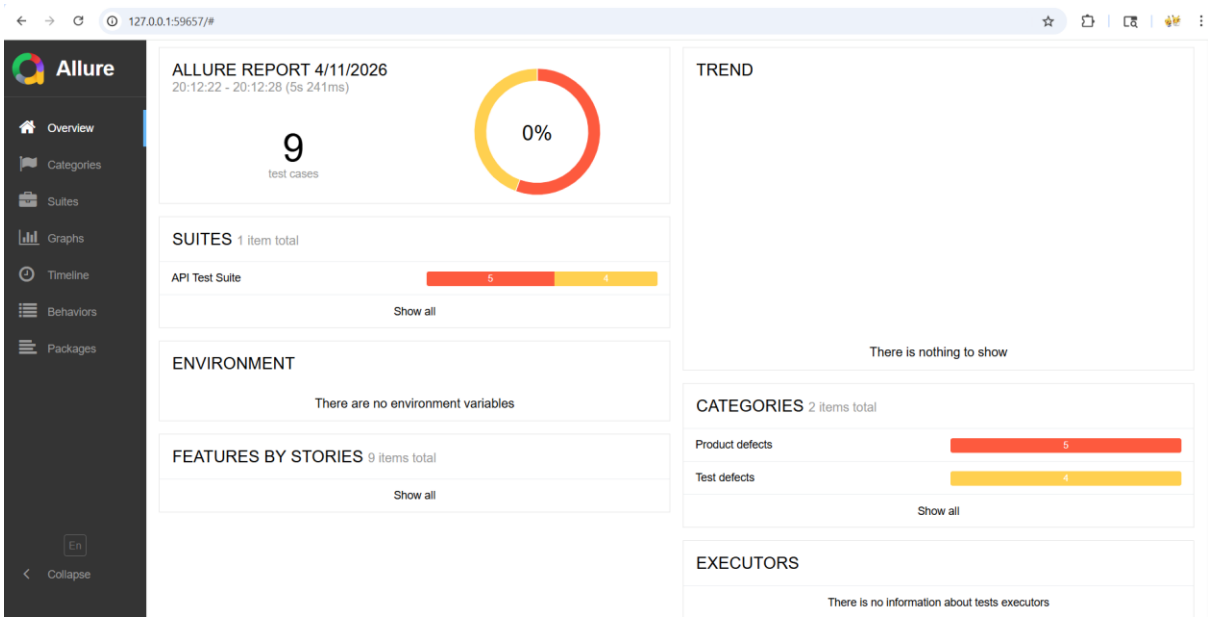
Git Bash terminal output showing API automation test execution with 11 failed scenarios, assertion errors in CommonSteps.java, and full failure summary:

```
MINGW64:/e/Veeva Prep/0_0_Veeva_Main_Project_Details/Merge/1_HybridAutomation_apitests/HybridAutomation_Precision/APIAutomation
        at com.framework.api.stepdefinitions.CommonSteps.the_response_status_code_should_be(CommonSteps.java:42)
        at ?.the response status code should be 200(file:///E:/Veeva%20Prep/0_0_Veeva_Main_Project_Details/Merge/1_HybridAutomation_apitests/HybridAutomation_Precision/APIAutomation/src/test/resources/features/
TC6_NegativeValidation.feature:29)

  And the response should contain empty search results                                    # com.framework.api.stepdefinitions.TC6_NegativeValidationSteps.the_response_should_contain_empty_search_result
s()
  And the response JSON schema should be valid for search results                        # com.framework.api.stepdefinitions.CommonSteps.the_response_json_schema_should_be_valid_for_search_results()
[ERROR] Tests run: 11, Failures: 11, Errors: 0, Skipped: 0, Time elapsed: 6.597 s <<< FAILURE! - in TestSuite
[ERROR] com.framework.api.runner.ApiTestRunner.runScenario["Get All Products List - TC1", "TC1 - Get All Products List"](1)  Time elapsed: 3.24 s  <<< FAILURE!
java.lang.AssertionError:

Expected: <999>
     but: was <200>
        at org.hamcrest.MatcherAssert.assertThat(MatcherAssert.java:20)
        at org.hamcrest.MatcherAssert.assertThat(MatcherAssert.java:6)
        at com.framework.api.stepdefinitions.CommonSteps.the_response_status_code_should_be(CommonSteps.java:42)
        at ?.the response status code should be 200(file:///E:/Veeva%20Prep/0_0_Veeva_Main_Project_Details/Merge/1_HybridAutomation_apitests/HybridAutomation_Precision/APIAutomation/src/test/resources/features/
TC1_GetAllProducts.feature:13)

[ERROR] com.framework.api.runner.ApiTestRunner.runScenario["Get Single Product Details by Search - TC2", "TC2 - Get Single Product Details"](2)  Time elapsed: 0.353 s  <<< FAILURE!
java.lang.AssertionError:

Expected: <999>
     but: was <200>
        at org.hamcrest.MatcherAssert.assertThat(MatcherAssert.java:20)
        at org.hamcrest.MatcherAssert.assertThat(MatcherAssert.java:6)
        at com.framework.api.stepdefinitions.CommonSteps.the_response_status_code_should_be(CommonSteps.java:42)
        at ?.the response status code should be 200(file:///E:/Veeva%20Prep/0_0_Veeva_Main_Project_Details/Merge/1_HybridAutomation_apitests/HybridAutomation_Precision/APIAutomation/src/test/resources/features/
TC2_GetSingleProduct.feature:13)

[ERROR] com.framework.api.runner.ApiTestRunner.runScenario["Get Single Product Details with Different Search Terms - TC2", "TC2 - Get Single Product Details"](3)  Time elapsed: 0.326 s  <<< FAILURE!
java.lang.AssertionError:

Expected: <999>
     but: was <200>
        at org.hamcrest.MatcherAssert.assertThat(MatcherAssert.java:20)
        at org.hamcrest.MatcherAssert.assertThat(MatcherAssert.java:6)
        at com.framework.api.stepdefinitions.CommonSteps.the_response_status_code_should_be(CommonSteps.java:42)
        at ?.the response status code should be 200(file:///E:/Veeva%20Prep/0_0_Veeva_Main_Project_Details/Merge/1_HybridAutomation_apitests/HybridAutomation_Precision/APIAutomation/src/test/resources/features/
TC2_GetSingleProduct.feature:21)

[ERROR] com.framework.api.runner.ApiTestRunner.runScenario["Get Single Product Details with Different Search Terms - TC2", "TC2 - Get Single Product Details"](4)  Time elapsed: 0.325 s  <<< FAILURE!
java.lang.AssertionError:

Expected: <999>
     but: was <200>
        at org.hamcrest.MatcherAssert.assertThat(MatcherAssert.java:20)
        at org.hamcrest.MatcherAssert.assertThat(MatcherAssert.java:6)
        at com.framework.api.stepdefinitions.CommonSteps.the_response_status_code_should_be(CommonSteps.java:42)
        at ?.the response status code should be 200(file:///E:/Veeva%20Prep/0_0_Veeva_Main_Project_Details/Merge/1_HybridAutomation_apitests/HybridAutomation_Precision/APIAutomation/src/test/resources/features/
TC2_GetSingleProduct.feature:21)

[ERROR] com.framework.api.runner.ApiTestRunner.runScenario["Get Single Product Details with Different Search Terms - TC2", "TC2 - Get Single Product Details"](5)  Time elapsed: 0.318 s  <<< FAILURE!
java.lang.AssertionError:

Expected: <999>
     but: was <200>
        at org.hamcrest.MatcherAssert.assertThat(MatcherAssert.java:20)
```

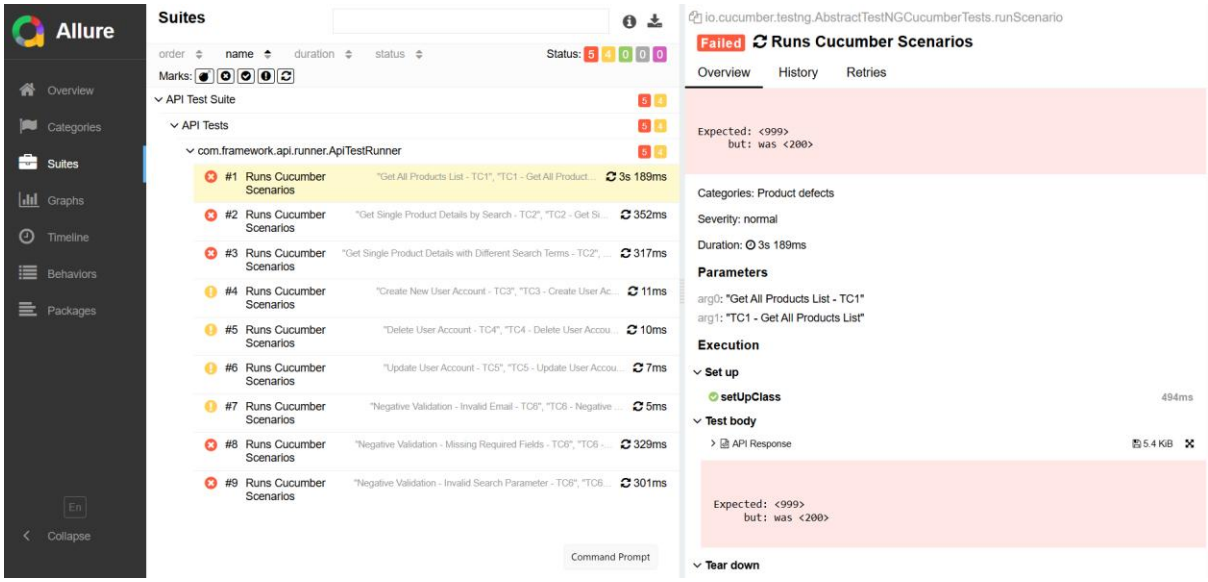
Git Bash terminal output showing API automation test execution with 11 failed scenarios due to HTTP 500 responses and ConfigReader NoSuchMethodException errors:

```
Expected: <999>
     but: was <200>
        at org.hamcrest.MatcherAssert.assertThat(MatcherAssert.java:20)
        at org.hamcrest.MatcherAssert.assertThat(MatcherAssert.java:6)
        at com.framework.api.stepdefinitions.CommonSteps.the_response_status_code_should_be(CommonSteps.java:42)
        at ?.the response status code should be 200(file:///E:/Veeva%20Prep/0_0_Veeva_Main_Project_Details/Merge/1_HybridAutomation_apitests/HybridAutomation_Precision/APIAutomation/src/test/resources/features/
TC6_NegativeValidation.feature:29)

[INFO]
[INFO] Results:
[INFO]
[ERROR] Failures:
[ERROR]   ApiTestRunner.runScenario » NoSuchMethod 'java.lang.String config.ConfigReader.get(java.lang.String, java.lang.String)'
[ERROR]   ApiTestRunner.runScenario » NoSuchMethod 'java.lang.String config.ConfigReader.get(java.lang.String, java.lang.String)'
[ERROR]   ApiTestRunner.runScenario
Expected: <999>
     but: was <200>
[ERROR]   ApiTestRunner.runScenario
Expected: <999>
     but: was <200>
[ERROR]   ApiTestRunner.runScenario
Expected: <999>
     but: was <200>
[ERROR]   ApiTestRunner.runScenario
Expected: <999>
     but: was <200>
[ERROR]   ApiTestRunner.runScenario
Expected: <999>
     but: was <200>
[ERROR]   ApiTestRunner.runScenario » NoSuchMethod 'java.lang.String config.ConfigReader.get(java.lang.String, java.lang.String)'
[ERROR]   ApiTestRunner.runScenario
Expected: <999>
     but: was <200>
[ERROR]   ApiTestRunner.runScenario
Expected: <999>
     but: was <200>
[ERROR]   ApiTestRunner.runScenario » NoSuchMethod 'java.lang.String config.ConfigReader.get(java.lang.String, java.lang.String)'
[INFO]
[ERROR] Tests run: 11, Failures: 11, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD FAILURE
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  9.549 s
[INFO] Finished at: 2026-04-11T20:12:28+05:30
[INFO] ------------------------------------------------------------------------
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire-plugin:3.0.0-M7:test (default-test) on project APIAutomation: There are test failures.
[ERROR]
[ERROR] Please refer to E:\Veeva Prep\0_0_Veeva_Main_Project_Details\Merge\1_HybridAutomation_apitests\HybridAutomation_Precision\APIAutomation\target\surefire-reports for the individual test results.
[ERROR] Please refer to dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and [date].dumpstream.
[ERROR] -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please read the following articles:
[ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException
```

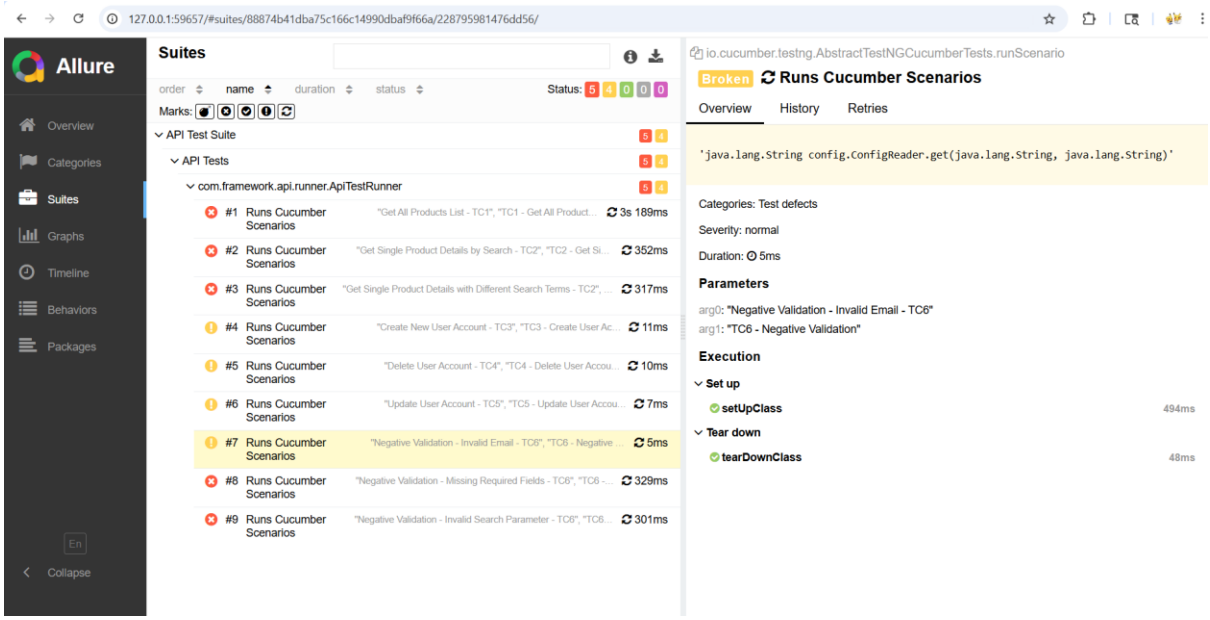
Allure report overview showing API Test Suite execution with nine test cases, categorized into product defects and test defects, and visual summary of pass/fail distribution:



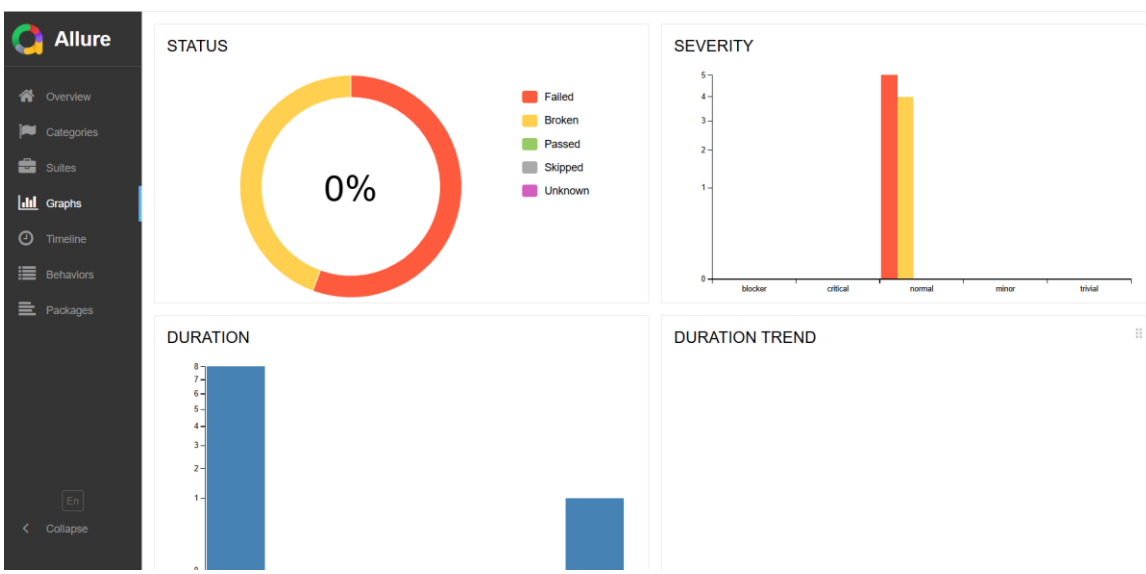
Allure report detailed view displaying failed API scenario ('Get All Products List - TC1'), with expected vs actual status code mismatch, defect categorization, parameters, and execution steps:



Allure report detailed view displaying broken API scenario ('Negative Validation - Invalid Email - TC6'), with ConfigReader exception, defect categorization, parameters, and execution steps:



Allure report graphs view summarizing API test execution with status distribution, severity breakdown, and duration analysis:



Conclusion

The combination of TestNG, Cucumber, Extent, and Allure reports ensures complete visibility of the automation framework. These reports help in tracking execution, identifying failures, and improving overall test quality.